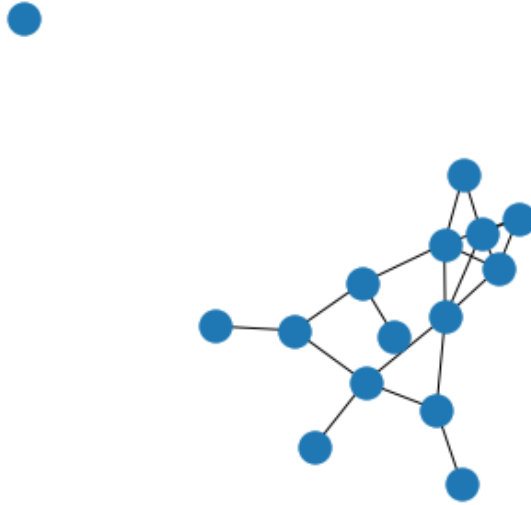
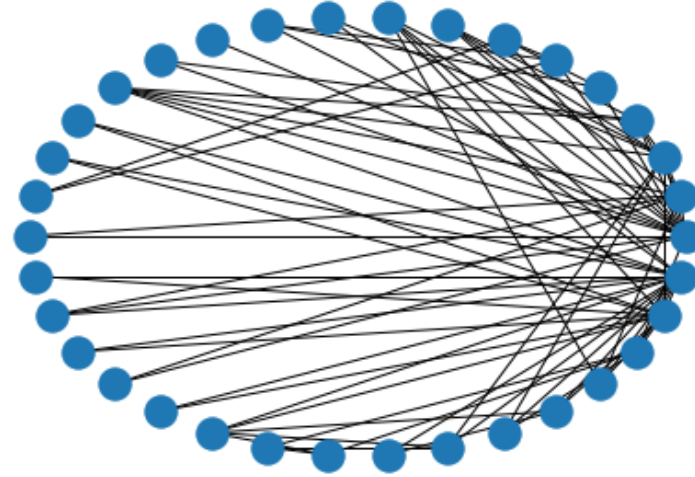


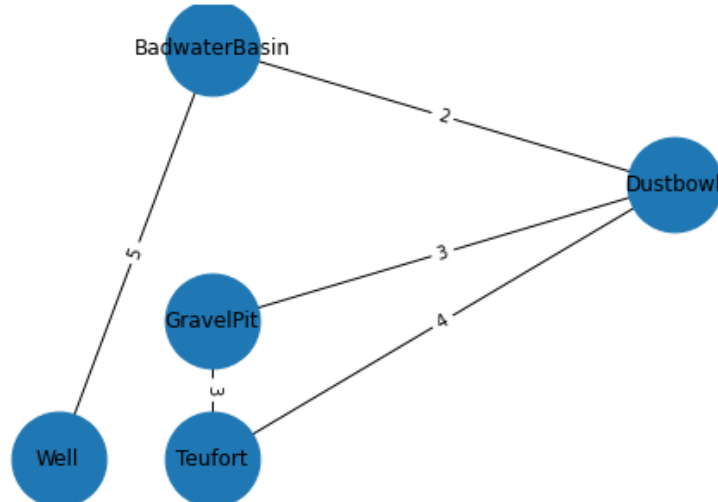
Erdos-Renyi Graph: 15 nodes, 0.2 Probability



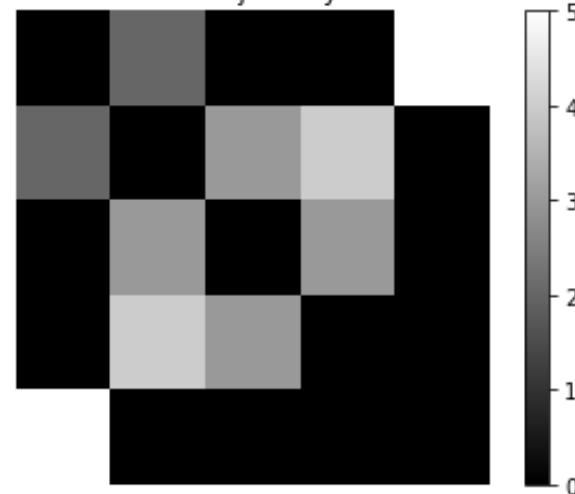
Zachary's Karate Club



Drive Times in the Badlands



Badlands Adjacency Matrix



Introduction to Networks

Objectives

- To create simple graphs and determine when the model might be useful
- To determine the degree distribution of a graph, and enumerate other properties of interest
- To calculate the adjacency matrix of a graph and explain its properties

Network Types

All parts of the universe are interconnected in one way or another – networks are everywhere and so the tools we use to manipulate and study them are of utmost importance. To begin our study of complex networks, we first create two simple graphs using Python's **NetworkX** library. **First, we create a simple graph, directed, and a weighted graph.**

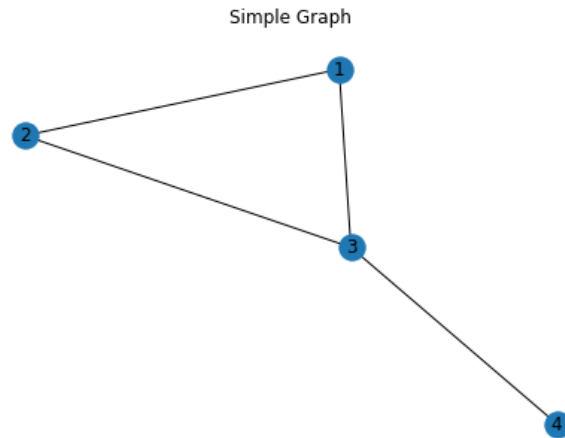


Fig 1. A simple graph with 4 nodes V (stands for vertices), and 4 edges E

A simple graph is characterized by nodes V , connected by reciprocal edges E . It just means *simple*, nothing more than connected vertices. We usually denote graphs as $G(V, E)$. Even with the simplicity of simple graphs, it's still powerful!

A directed graph is different from a simple graph only in the lack of reciprocity in its edges. This one way direction is usually denoted as an arrow. These graphs have a reverse graph.

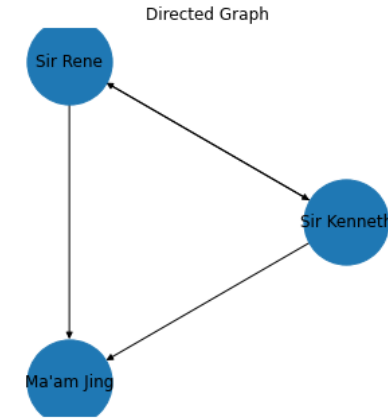


Fig 2. A directed graph with $|V| = 3$, and $|E| = 4$

A weighted graph has attached weights to each edge which can represent any information we want.

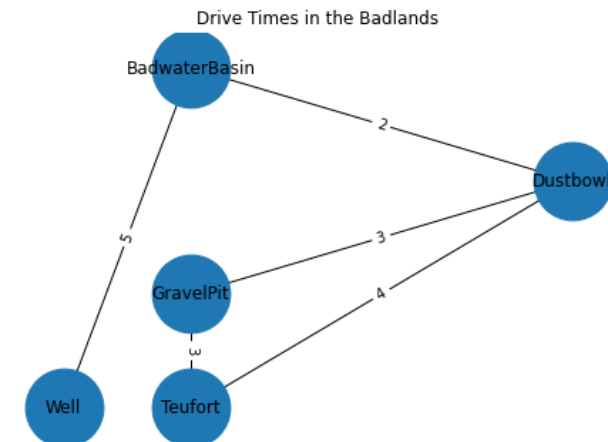


Fig 3. A weighted network, with weights denoting hours of driving

Other Types and Degree

There are many other network types used in complexity science, some are listed below:

1. **Bipartite Networks** – Graphs that have two types of nodes. Edges can only connect nodes of different types. If we have n of nodes then we generalize to an **n -partite** network.
2. **Multigraphs** – We allow multiple (parallel) edges to connect nodes. I'm quite familiar with this as I use this in my research because the library I use represents road networks to multigraphs!
3. **Multilayer Networks** – Different layers representing how different nodes can connect in different ways. Each layer can connect with each other (many to many).
4. **Signed Networks** – Multilayer network with just two layers, one representing “friendship” and one representing “enmity”.
5. **Hypergraphs** – Contains hyperedges that can connect more than two nodes at the same time.
6. **Dynamic Graphs** – Contains temporal information on nodes and edges to model evolving systems.

Now that we know different types of network, we move on to actually working with them. There are many properties we usually measure concerning graph, but one of the most important is **degree**. This is a property of a node that tells us how connected that particular node is to others. **In Fig 4. below, I load Zachary's karate club then determine each node's degree. We then use the weights stored in each edge to generate a degree distribution in Fig 5.**

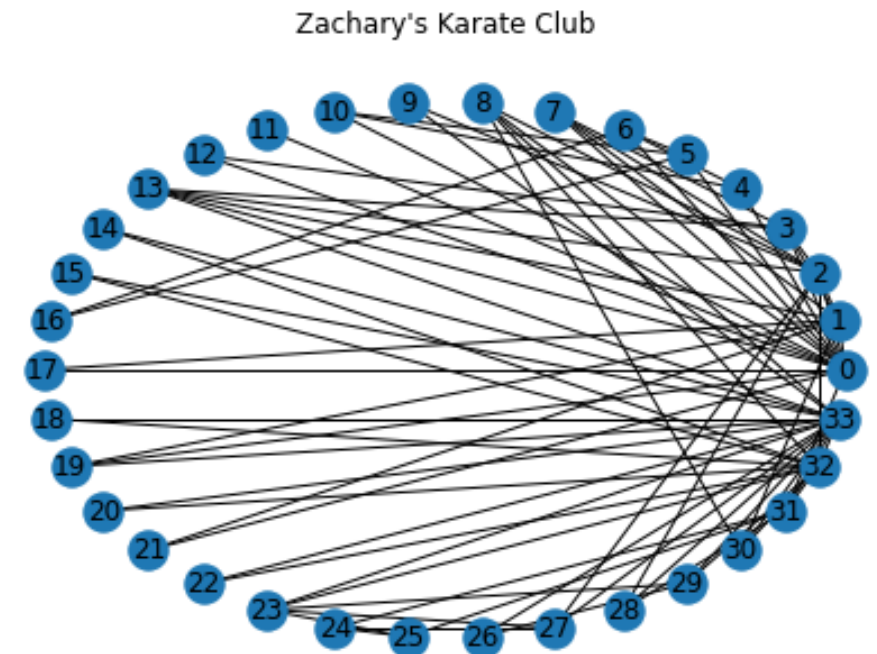


Fig 4. Zachary's karate club comprising 33 nodes and weighted edges

Degree Distribution

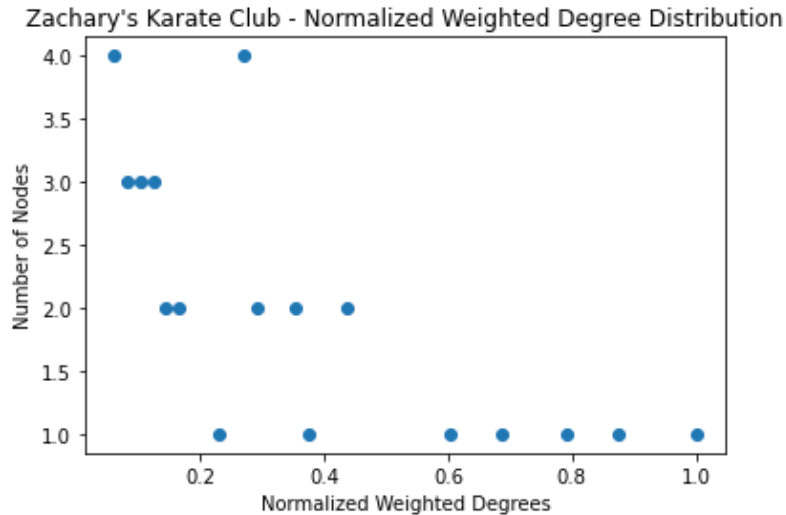


Fig 5. Degree distribution of Zachary's karate club

Since the network is weighted, the contribution of each edge to a node's degree are different unlike in a simple graph. Since Zachary's karate club is not a complex network, its degree distribution appears discrete and does not follow typical trends such as a power law or a log-scale.

Networkx also provides functions to calculate the clustering coefficient of each node, and shortest path length using **Dijkstra's** or **Floyd-Warshall's algorithm**. The former shortest-path algorithm is of course used in origin-destination analysis and the latter for analyzing the whole network

Random Graphs (Erdos-Rényi)

To explore various network properties, one must first create a network (amazing, I know). Suppose we want to model plant epidemics and how they spread through out time using a novel model. One way to generate your network in this case is by making an **Erdos-Rényi (ER) graph**, particularly the **$G(n,p)$** kind.

In the $G(n,p)$ model, a graph is generated by connecting nodes based on a probability p . Each edge is included independently of any other edge. In Fig 6., I generate these graphs...

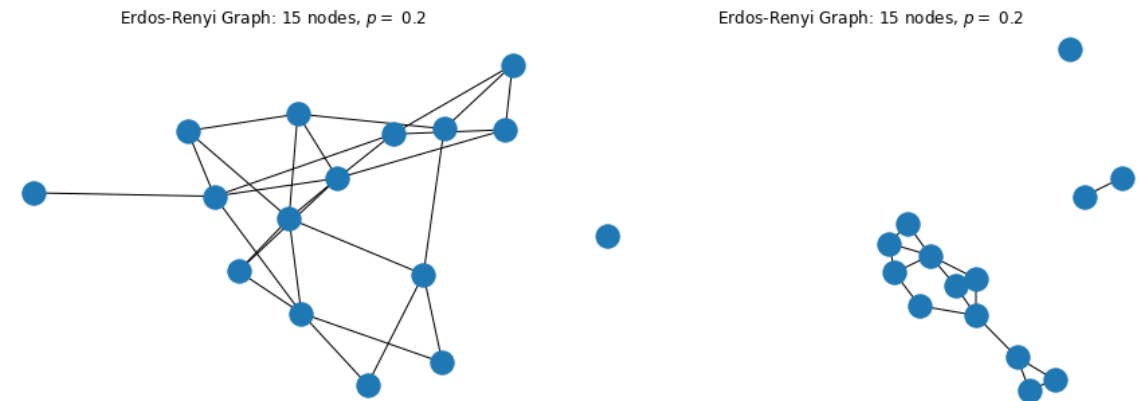


Fig 6. Randomly generated ER graphs with the same parameters

There are other types of graphs we can use, such as random geometric etc., but it is up to the scenario we model what to use.

Adjacency Matrix

The adjacency matrix is a basic representation of a graph as a matrix (I know, shocking). To explore its properties, let us first create a graph. **For this example, I make a weighted graph whose nodes are dependent on position shown in Fig 7.**

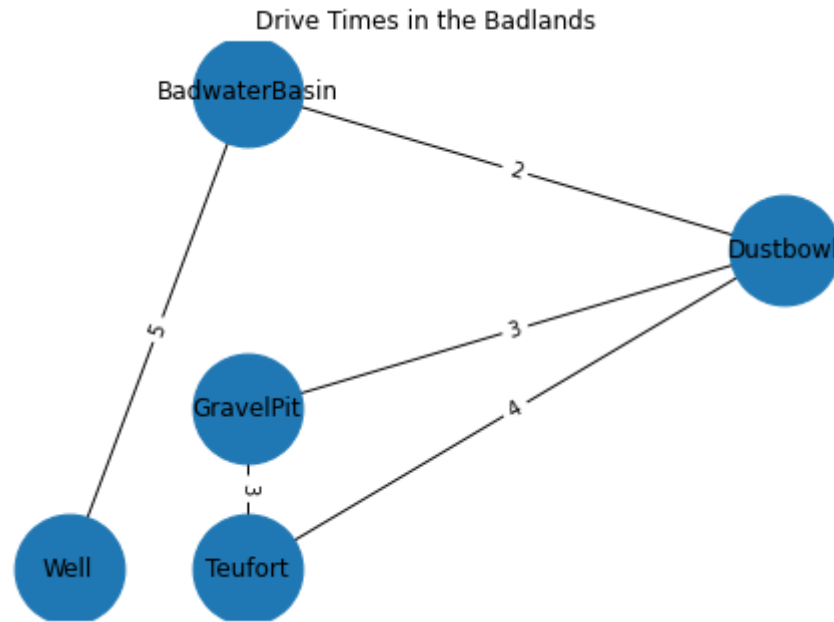


Fig 7. A weighted graph with 5 nodes. The positions of the nodes were defined manually. The network is non-directed and has no self-loops

Now we can generate the adjacency matrix! Each row/column corresponds to a node, and the elements of the matrix represent each connection as shown in Fig 8.

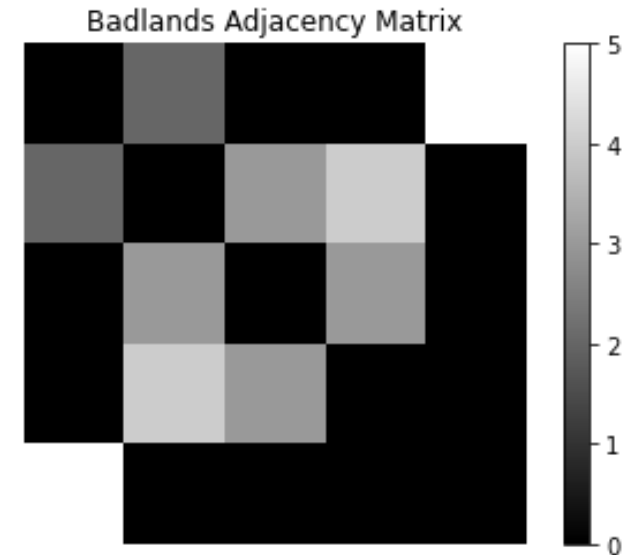


Fig 8. Adjacency matrix of the network in Fig 7. Note that it is a square matrix that is mirrored with respect to the diagonal. Moreover, this diagonal is comprised of 0's

We highlight some qualities of this adjacency matrix:

1. **It is a square matrix** since it is not an n-partite network. The graph is just a simple weighted graph.
2. **The diagonal is a 0 diagonal** because there are no self-loops present in the network
3. **The matrix is mirrored with respect to the diagonal** because all edges have reciprocity. If this were a directed graph, it would be a different story.

With this matrix, we can perform various operations on a network, like find its complement, or its Laplacian, etc...

Key Takeaways

- There are many ways to construct a network, and choosing which one to build relies on the physical phenomena we want to model
- A network has many properties of interest that we usually want to measure such as its degree distribution, the shortest path between nodes, and the clustering coefficient of its nodes
- One can generate random graphs as synthetic data in order to explore these properties.
- A network can be represented by an adjacency matrix, which can be acted upon by operators to transform the network.

Reflection

The reference for this activity was really interesting – I liked the writing style of the author and how ideas are introduced. However, I had little time to touch on all the details of each chapter as the semester nears its end. In fact out of all the activities, I was forced to spend the least time on this one (also because of the neural network module). It is a good thing that I'm already familiar with many of the concepts in the reference due to being part of the Complex Systems Laboratory of the institute! When conducting preliminary work for my (future) research, I needed to learn how complex networks behave and what properties we can extract from them. It's cool to know I'm actually learning things ahahahaha...

I asked the other class about what they did for this activity and based on their stories, I was able to demonstrate more techniques that what was required of them. Because of this, I think I deserve a 100/100. I know I wasn't able to include some network tools and concepts, but as an introductory activity I think I covered more than enough.

Reference

[1] M. Coscia. The Atlas for the Aspiring Network Scientist. 2021. Chapters 3, 4, 6, 9, 10, 11, and 14. <https://www.networkatlas.eu>